6.4 Anchor boxes

- object detection algorithms usually sample a large number of regions in the input image, determine whether these regions contain objects of interest, and adjust the boundaries of the regions so as to predict the *ground-truth bounding boxes* of the objects more accurately
- different models may adopt different region sampling schemes
- here we introduce one of such methods: it generates multiple bounding boxes with varying scales and aspect ratios, centered on each pixel
- these bounding boxes are called *anchor boxes*
- we will design an object detection model based on anchor boxes in Section 6.6

6.4 Anchor boxes

- suppose that the input image has a height of h and a width of w
- we generate anchor boxes with different shapes centered on each pixel of the image
- let the *scale* be $s \in (0, 1]$ and the *aspect ratio* (ratio of width to height) be $r > 0$
- then, the width and height of the anchor box are $ws\sqrt{r}$ and $hs/\sqrt{r}$, respectively
- note that when the center position is given, an anchor box with known width and height is determined

6.4 Anchor boxes

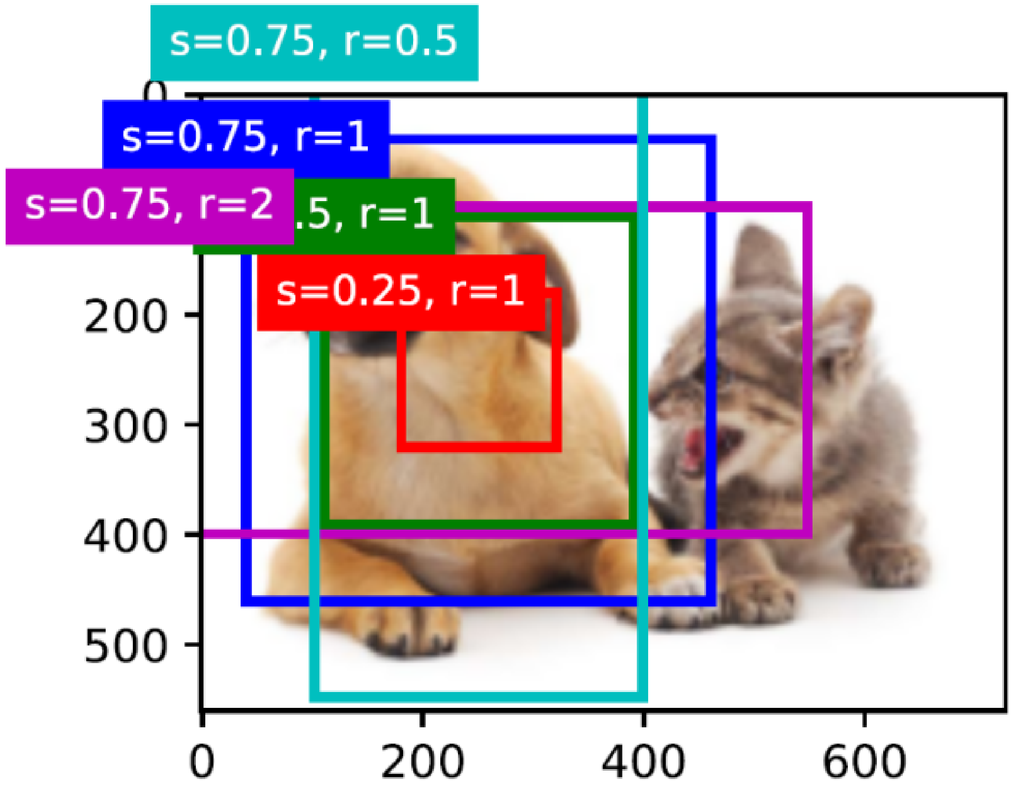
- to generate multiple anchor boxes with different shapes, let us set a series of scales $s_1, \ldots, s_n$ and a series of aspect ratios $r_1, \ldots, r_m$
- when using all the combinations of these scales and aspect ratios with each pixel as the center, the input image will have a total of $whnm$ anchor boxes
- although these anchor boxes may cover all the ground-truth bounding boxes, the computational complexity is generally too high
- in practice, we can only consider those combinations containing s_1 or r_1:

$$(s_1, r_1), (s_1, r_2), \ldots, (s_1, r_m), (s_2, r_1), (s_3, r_1), \ldots, (s_n, r_1).$$

- that is to say, the number of anchor boxes centered on the same pixel is $n + m - 1$
- for the entire input image, we will generate a total of $wh(n + m - 1)$ anchor boxes

6.4 Anchor boxes

- for our example, we can draw all the anchor boxes centered on $(250, 250)$ in the image
- as we can see, the blue anchor box with a scale of 0.75 and an aspect ratio of 1 well surrounds the dog in the image



6.4 Anchor boxes

- we just mentioned that an anchor box “well” surrounds the dog in the image
- if the ground-truth bounding box of the object is known, how can “well” here be quantified?
- intuitively, we can measure the similarity between the anchor box and the ground-truth bounding box
- we know that the *Jaccard index* can measure the similarity between two sets
- given sets $\mathcal{A}$ and $\mathcal{B}$, their Jaccard index is the size of their intersection divided by the size of their union:

$$J(\mathcal{A}, \mathcal{B}) = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A} \cup \mathcal{B}|}.$$

6.4 Anchor boxes

- in fact, we can consider the pixel area of any bounding box as a set of pixels
- in this way, we can measure the similarity of the two bounding boxes by the Jaccard index of their pixel sets
- for two bounding boxes, we usually refer to their Jaccard index as the *intersection over union* (*IoU*), which is the ratio of their intersection area to their union area, as shown in Figure 2
- the range of an IoU is between 0 and 1: 0 means that two bounding boxes do not overlap at all, while 1 indicates that the two bounding boxes are equal

6.4 Anchor boxes

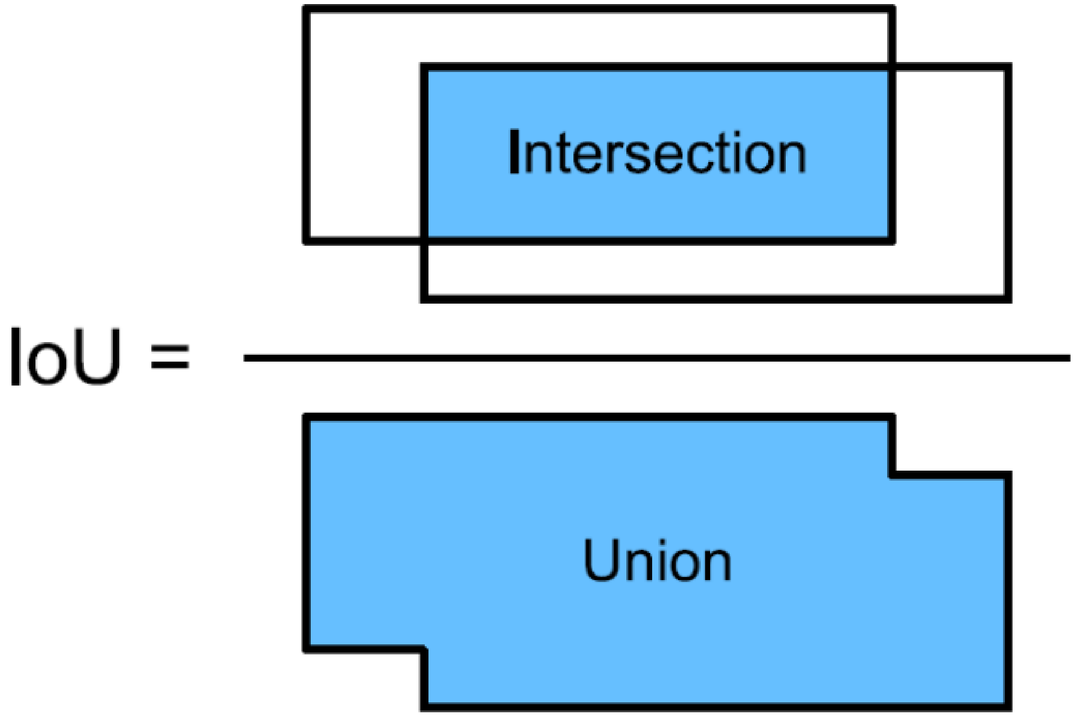


Figure 2: IoU is the ratio of the intersection area to the union area of two bounding boxes.

6.4 Anchor boxes

- we will use IoU to measure the similarity between anchor boxes and ground-truth bounding boxes, and between different anchor boxes
- in a training dataset, we consider each anchor box as a training example
- in order to train an object detection model, we need *class* and *offset* labels for each anchor box, where the former is the class of the object relevant to the anchor box, and the latter is the offset of the ground-truth bounding box relative to the anchor box
- during prediction, for each image, we generate multiple anchor boxes, predict classes and offsets for all the anchor boxes, adjust their positions according to the predicted offsets to obtain the predicted bounding boxes, and finally only output those predicted bounding boxes that satisfy certain criteria

6.4 Anchor boxes

- in general, an object detection training set comes with labels for locations of *ground-truth bounding boxes* and classes of their surrounded objects
- to label any generated *anchor box*, we refer to the labeled location and class of its *assigned* ground-truth bounding box that is closest to the anchor box
- in the following, we describe an algorithm for assigning closest ground-truth bounding boxes to anchor boxes
- given an image, suppose that the anchor boxes are $A_1, A_2, \ldots, A_{n_a}$, and the ground-truth bounding boxes are $B_1, B_2, \ldots, B_{n_b}$, where $n_a \geq n_b$
- let us define a matrix $\boldsymbol{X} \in \mathbb{R}^{n_a \times n_b}$, whose element x_{ij} in the ith row and jth column is the IoU of the anchor box A_i and the ground-truth bounding box B_j

6.4 Anchor boxes

- the algorithm consists of the following steps:
  1. Find the largest element in matrix $\boldsymbol{X}$, and denote its row and column indices as i_1 and j_1, respectively. Then, the ground-truth bounding box B_{j_1} is assigned to the anchor box A_{i_1}. This is quite intuitive, because A_{i_1} and B_{j_1} are the closest among all the pairs of anchor boxes and ground-truth bounding boxes. After the first assignment, discard all the elements in the i_1th row and the j_1th column in matrix $\boldsymbol{X}$.
  2. Find the largest of the remaining elements in matrix $\boldsymbol{X}$, and denote its row and column indices as i_2 and j_2, respectively. We assign ground-truth bounding box B_{j_2} to anchor box A_{i_2}, and discard all the elements in the i_2th row and the j_2th column in matrix $\boldsymbol{X}$.
  3. At this point, elements in two rows and two columns in matrix $\boldsymbol{X}$ have been discarded. We proceed until all elements in n_b columns in matrix $\boldsymbol{X}$ are discarded. At this time, we have assigned a ground-truth bounding box to each of n_b anchor boxes.
  4. Only traverse through the remaining $n_a - n_b$ anchor boxes. For example, given any anchor box A_i, find the ground-truth bounding box B_j with the largest IoU with A_i throughout the ith row of matrix $\boldsymbol{X}$, and assign B_j to A_i only if this IoU is greater than a predefined threshold.

6.4 Anchor boxes

- let us illustrate the above algorithm using a concrete example
- as shown in Figure 3 (left), assuming that the maximum value in matrix $\boldsymbol{X}$ is x_{23}, we assign the ground-truth bounding box B_3 to the anchor box A_2
- then, we discard all the elements in row 2 and column 3 of the matrix, find the largest x_{71} in the remaining elements (shaded area), and assign the ground-truth bounding box B_1 to the anchor box A_7
- next, as shown in Figure 3 (middle), discard all the elements in row 7 and column 1 of the matrix, find the largest x_{54} in the remaining elements (shaded area), and assign the ground-truth bounding box B_4 to the anchor box A_5
- finally, as shown in Figure 3 (right), discard all the elements in row 5 and column 4 of the matrix, find the largest x_{92} in the remaining elements (shaded area), and assign the ground-truth bounding box B_2 to the anchor box A_9
- after that, we only need to traverse through the remaining anchor boxes A_1, A_3, A_4, A_6, A_8 and determine whether to assign them ground-truth bounding boxes according to the threshold

6.4 Anchor boxes

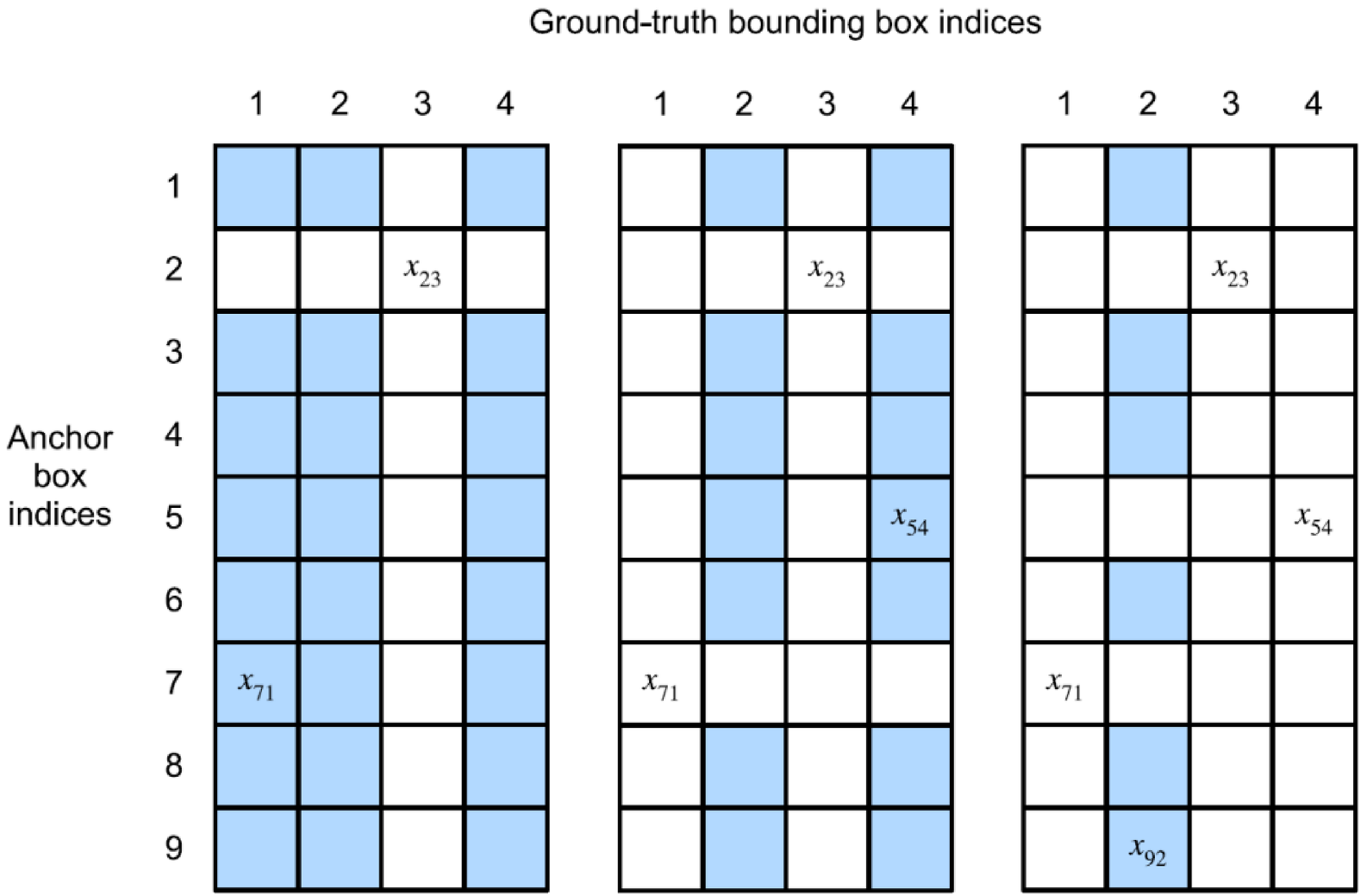


Figure 3: Assigning ground-truth bounding boxes to anchor boxes.

6.4 Anchor boxes

- now, we can label the class and offset for each anchor box
- suppose that an anchor box A is assigned a ground-truth bounding box B
- on one hand, the class of the anchor box A will be labeled as that of B
- on the other hand, the offset of the anchor box A will be labeled according to the relative position between the central coordinates of B and A, together with the relative size between these two boxes
- given varying positions and sizes of different boxes in the dataset, we can apply transformations to those relative positions and sizes that may lead to more uniformly distributed offsets that are easier to fit
- here, we describe a common transformation; given the central coordinates of A and B as (x_a, y_a) and (x_b, y_b), their widths as w_a and w_b, and their heights as h_a and h_b, respectively
- we may label the offset of A as:

$$\left(\frac{\frac{x_b - x_a}{w_a} - \mu_x}{\sigma_x}, \frac{\frac{y_b - y_a}{h_a} - \mu_y}{\sigma_y}, \frac{\log \frac{w_b}{w_a} - \mu_w}{\sigma_w}, \frac{\log \frac{h_b}{h_a} - \mu_h}{\sigma_h} \right),$$

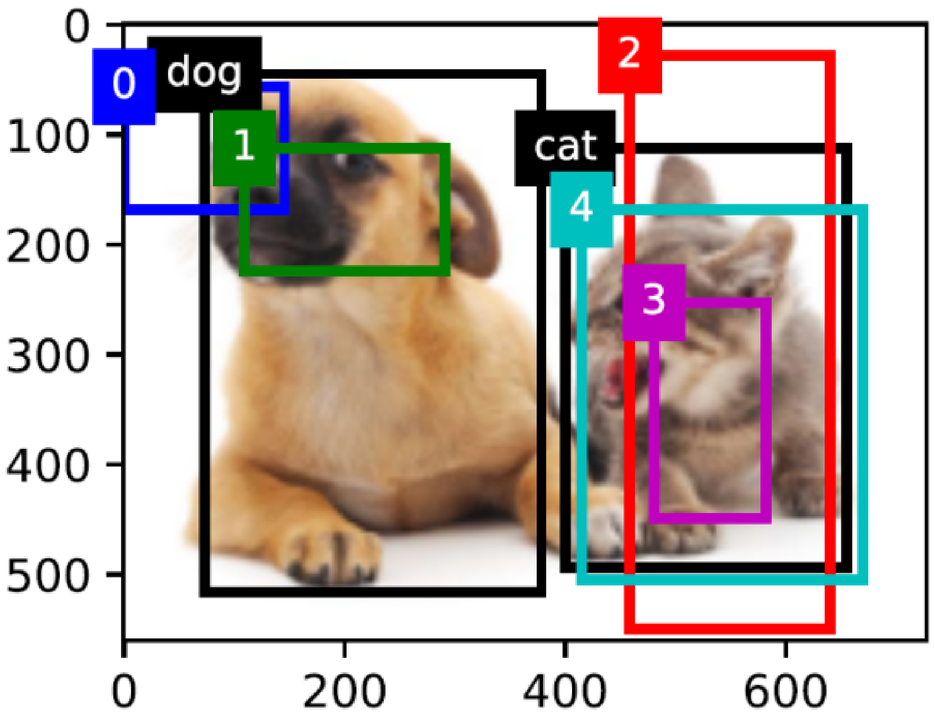
where the default values of the constants are: $\mu_x = \mu_y = \mu_w = \mu_h = 0$, $\sigma_x = \sigma_y = 0.1$, and $\sigma_w = \sigma_h = 0.2$

6.4 Anchor boxes

- if an anchor box is not assigned a ground-truth bounding box, we just label the class of the anchor box as “background”
- anchor boxes whose classes are background are often referred to as *negative* anchor boxes, and the rest are called *positive* anchor boxes
- let us illustrate anchor box labeling via a concrete example
- we define ground-truth bounding boxes for the dog and cat in the loaded image, where the first element is the class (1 for dog and 2 for cat) and the remaining four elements are the (x, y)-axis coordinates at the upper-left corner and the lower-right corner

6.4 Anchor boxes

- we also construct five anchor boxes to be labeled using the coordinates of the upper-left corner and the lower-right corner: $A_0, \ldots, A_4$ (the index starts from 0)
- then, we plot these ground-truth bounding boxes and anchor boxes in the image



6.4 Anchor boxes

- we can label classes and offsets of these anchor boxes based on the ground-truth bounding boxes for the dog and cat
- in this example, indices of the background, dog, and cat classes are 0, 1, and 2, respectively
- let us analyze the class labels that will be assigned based on anchor box and ground-truth bounding box positions in the image
- first, among all the pairs of anchor boxes and ground-truth bounding boxes, the IoU of the anchor box A_4 and the ground-truth bounding box of the cat is the largest
- thus, the class of A_4 is labeled as the cat

6.4 Anchor boxes

- taking out pairs containing A_4 or the ground-truth bounding box of the cat, among the rest, the pair of the anchor box A_1 and the ground-truth bounding box of the dog has the largest IoU
- so the class of A_1 is labeled as the dog
- next, we need to traverse through the remaining three unlabeled anchor boxes: A_0, A_2, and A_3
- for A_0, the class of the ground-truth bounding box with the largest IoU is the dog, but the IoU is below the predefined threshold (0.5), so the class is labeled as background
- for A_2, the class of the ground-truth bounding box with the largest IoU is the cat and the IoU exceeds the threshold, so the class is labeled as the cat
- for A_3, the class of the ground-truth bounding box with the largest IoU is the cat, but the value is below the threshold, so the class is labeled as background

6.4 Anchor boxes

- since we do not care about background detection, offsets of the negative class should not affect the objective function
- through element-wise multiplications, zeros in a mask variable will filter out negative class offsets before calculating the objective function
- during prediction, we generate multiple anchor boxes for the image, and predict classes and offsets for each of them
- a *predicted bounding box* is thus obtained according to an anchor box, with its predicted offset
- when there are many anchor boxes, many similar (with significant overlap) predicted bounding boxes can be potentially output for surrounding the same object
- to simplify the output, we can merge similar predicted bounding boxes that belong to the same object by using *non-maximum suppression* (NMS)

6.4 Anchor boxes

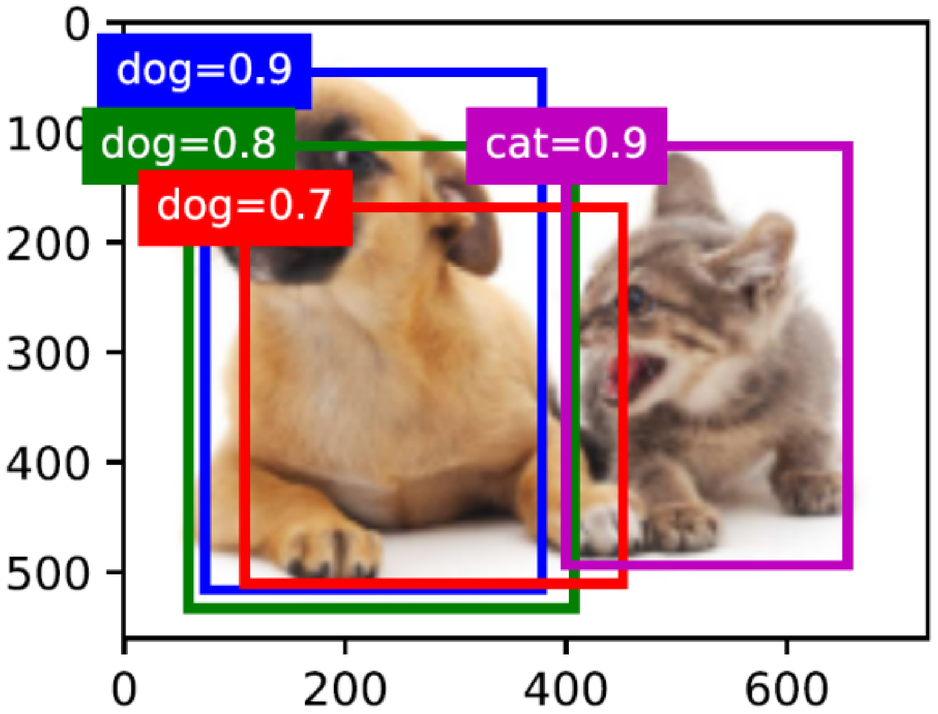
- here is how non-maximum suppression works
- for a predicted bounding box B, the object detection model calculates the predicted likelihood for each class
- denoting by p the largest predicted likelihood, the class corresponding to this probability is the predicted class for B
- specifically, we refer to p as the *confidence* (score) of the predicted bounding box B
- on the same image, all the predicted non-background bounding boxes are sorted by confidence, in descending order, to generate a list L

6.4 Anchor boxes

- then, we manipulate the sorted list L in the following steps:
  1. Select the predicted bounding box B_1 with the highest confidence from L as a basis, and remove all non-basis predicted bounding boxes whose IoU with B_1 exceeds a predefined threshold ϵ from L. At this point, L keeps the predicted bounding box with the highest confidence, but drops others that are too similar to it. In a nutshell, those with *non-maximum* confidence scores are *suppressed*.
  2. Select the predicted bounding box B_2 with the second highest confidence from L as another basis, and remove from L all non-basis predicted bounding boxes whose IoU with B_2 exceeds ϵ.
  3. Repeat the above process until all the predicted bounding boxes in L have been used as a basis. At this time, the IoU of any pair of predicted bounding boxes in L is below the threshold ϵ; thus, no pair is too similar with each other.
  4. Output all the predicted bounding boxes in the list L.

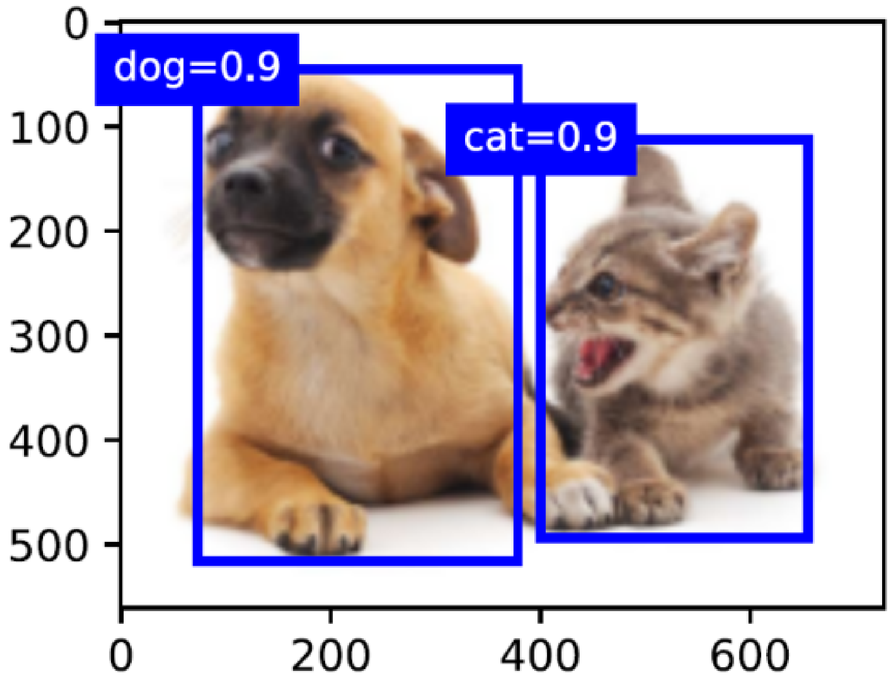
6.4 Anchor boxes

- now, let us apply the above methods to a concrete example with four anchor boxes
- for simplicity, we assume that the predicted offsets are all zeros
- this means that the predicted bounding boxes are anchor boxes
- for each class among the background, dog, and cat, we also define its predicted likelihood
- we can plot these predicted bounding boxes with their confidence on the image



6.4 Anchor boxes

- now, we can perform non-maximum suppression, where the threshold is set to 0.5
- after removing those predicted bounding boxes of the background class, we can output the final predicted bounding box kept by non-maximum suppression



- in practice, we can remove predicted bounding boxes with lower confidence even before performing non-maximum suppression, thereby reducing computation in this algorithm
- we may also post-process the output of non-maximum suppression, for example, by only keeping results with higher confidence in the final output